601.433/633 Introduction to Algorithms **Fall 2026**
Homework #7 **Due:** November 5, 2026, 11:59pm

Remember: this is an oral presentation homework! You are required to work in groups of three, and may collaborate only with other students in your group. We encourage you to write up solutions, but your grade will be based entirely on your presentation. At your presentation, the grader will randomly assign each student to present a problem. You should be able to create a group and sign up for a slot using Canvas. The deadline for signing up for a presentation slot is 5pm Wed Nov 4.

You may use the internet or LLMs to look up formulas, definitions, concepts, etc., but may not simply look up the answers online or ask an LLM.

1 Faster Shortest Paths

Let $G = (V, E)$ be a directed graph with lengths $\ell : E \to \mathbb{R}$ with no negative-length cycles. Let $v \in V$, and let α denote the maximum, over all $u \in V$, of the number of edges on a shortest path from v to u (where the shortest-path is defined with respect to the weights). Given G, w, and v (but not α), give an algorithm that computes shortest path distances from v to all other nodes in $O(m\alpha)$ time. You may assume that $m = \Omega(n)$. Prove correctness and running time.
Solution.

- **Subproblem**
  Let $\mathrm{OPT}(u, k)$ be the minimum length of a path from the start vertex v to u which uses at most k edges. We want $\mathrm{OPT}(u, k)$ for large enough k, which is the shortest path distance $d(v, u)$.

- **Recurrence**

$$\mathrm{OPT}(u,k) = \begin{cases} 0 & \text{if } k = 0,\ u = v, \\ \infty & \text{if } k = 0,\ u \neq v, \\ \min\left\{\mathrm{OPT}(u, k-1),\ \min\limits_{w:(w,u)\in E}\{\mathrm{OPT}(w, k-1) + \ell(w,u)\}\right\} & \text{otherwise.} \end{cases}$$

  A path to u with at most k edges either uses at most $k-1$ edges, or its last edge is some (w, u) and the rest is a path to w with at most $k-1$ edges.

- **Algorithm**
  This is the Bellman-Ford algorithm, but we do not run the outer loop $n-1$ times. Since we do not know α, we stop as soon as one round of the outer loop changes nothing.

  $M(v, 0) \leftarrow 0$, and $M(u, 0) \leftarrow \infty$ for all $u \neq v$
  $k \leftarrow 0$
  repeat
      $k \leftarrow k + 1$
      for $u \in V$ **do**
          $M(u,k) \leftarrow \min\left\{M(u, k-1),\ \min_{w:(w,u)\in E}\{M(w, k-1) + \ell(w,u)\}\right\}$
      end for

8: **until** $M(u,k) = M(u,k-1)$ for all $u \in V$
9: **return** $M(\cdot,k)$

We will show that the outer loop runs at most $\alpha+1$ times.

Correctness.

- **Stopping rule**
  By the recurrence, $M(\cdot,k+1)$ is computed only from $M(\cdot,k)$, by exactly the same rule as $M(\cdot,k)$ is computed from $M(\cdot,k-1)$. So if $M(\cdot,k) = M(\cdot,k-1)$, the same input gives the same output, and $M(\cdot,k+1) = M(\cdot,k)$. Repeating this argument (by induction), the values never change again. This means that when the algorithm stops at round k, $M(x,k) = \mathrm{OPT}(x,\beta)$ for every $\beta \geq k$, and by the Claim (taking $\beta \geq \max\{k,\alpha\}$) this is $d(v,x)$. Also, by the Claim, $M(\cdot,\alpha+1) = M(\cdot,\alpha)$, so the algorithm stops at some round $k \leq \alpha+1$.

Therefore, the algorithm returns the correct shortest path distances.

Running Time.

- **One round of the outer loop**
  For each u, we look at every incoming edge (w,u) once, so over all u we look at every edge once. This takes $O(n+m)$. Checking whether $M(u,k) = M(u,k-1)$ for all u takes $O(n)$. Since $m = \Omega(n)$, one round takes $O(m)$.

- **Number of rounds**
  By the stopping rule, the outer loop runs at most $\alpha+1 \leq 2\alpha$ times (when $\alpha \geq 1$).

Therefore, the total running time is $O(m) \cdot O(\alpha) = O(m\alpha)$.

2 Multi-Set Shortest Paths

Let $G = (V,E)$ be a directed graph with weighted edges; edge weights can be positive, negative, or zero. There are no negative-weight cycles. Suppose the vertices of G are partitioned into k disjoint subsets $V_1, V_2, \ldots, V_k$; that is, every vertex of G belongs to exactly one subset V_i. For each $i,j \in [k]$, let $\delta(i,j)$ denote the minimum shortest-path distance between vertices in V_i and vertices in V_j:

$$\delta(i,j) = \min\{d(v_i,v_j) \mid v_i \in V_i \text{ and } v_j \in V_j\}$$

Design an algorithm to compute $\delta(i,j)$ for all $i,j \in [k]$ that runs in time $O(mn + kn\log n)$. You may assume that $m = \Omega(n)$. Prove correctness and running time.

Idea. For each group V_i, we make a portal vertex s_i which leads to every vertex of V_i for free, so that each group can be handled as a single source.

Solution.

- **Step 1 (Reweighting, as in Johnson's algorithm)**
  Construct $G' = (V',E')$, where $V' = V \cup \{q\}$ for a new vertex q, and $E' = E \cup \{(q,x) : x \in V\}$ with $w(q,x) = 0$ for all $x \in V$. Run Bellman-Ford on G' from q, and let $h(x) = d(q,x)$ for all $x \in V$. Define the new weight

  $$\hat{w}(u,v) = w(u,v) + h(u) - h(v) \quad \text{for all } (u,v) \in E,$$

  and let $\hat{d}$ denote the shortest path distance with respect to $\hat{w}$.

- **Step 2 (Super source for each group)**
  For each $i \in [k]$, construct $G_i = (V \cup \{s_i\},\ E \cup \{(s_i, x) : x \in V_i\})$ for a new vertex s_i, with $\hat{w}(s_i, x) = -h(x)$ for all $x \in V_i$. Run Dijkstra's algorithm on G_i from s_i with weight $\hat{w}$, and let $D_i(y)$ be the distance from s_i to y for all $y \in V$.

- **Step 3**
  For each $i, j \in [k]$, return
  $$\delta(i, j) = \min_{y \in V_j} \{D_i(y) + h(y)\}.$$

Correctness.

- **Step 1**
  q has no incoming edges, so G' has no negative-weight cycle and h is well defined. By the triangle inequality, $h(v) \leq h(u) + w(u, v)$, so $\hat{w} \geq 0$. For any path p from x to y, the sum telescopes to $\hat{w}(p) = w(p) + h(x) - h(y)$, so $\hat{d}(x, y) = d(x, y) + h(x) - h(y)$.

- **Step 2**
  $h(x) \leq w(q, x) = 0$, so $\hat{w}(s_i, x) = -h(x) \geq 0$, and Dijkstra's algorithm works correctly. Since s_i has no incoming edges, every path from s_i to y is one edge (s_i, x) with $x \in V_i$, followed by a path from x to y in G. So
  $$D_i(y) = \min_{x \in V_i} \left\{-h(x) + \hat{d}(x, y)\right\} = \min_{x \in V_i} d(x, y) - h(y).$$

- **Step 3**
  By Step 2, $\min_{y \in V_j} \{D_i(y) + h(y)\} = \min_{y \in V_j} \min_{x \in V_i} d(x, y) = \delta(i, j)$.

Running Time.

- **Step 1**
  G' has $n + 1$ vertices and $m + n$ edges, so Bellman-Ford takes $O((n + 1)(m + n)) = O(mn)$, since $m = \Omega(n)$. Computing $\hat{w}$ takes $O(m)$.

- **Step 2**
  Each G_i has $n + 1$ vertices and at most $m + n$ edges, so Dijkstra's algorithm with a Fibonacci heap takes $O(m + n \log n)$. Over all i, this takes $O(km + kn \log n)$. Since $V_1, \ldots, V_k$ partition V, $k \leq n$, so $km \leq mn$.

- **Step 3**
  For a fixed i, one scan over all $y \in V$ gives $\delta(i, j)$ for all j, in $O(n)$. Over all i, this takes $O(kn)$.

Therefore, the total running time is $O(mn) + O(mn + kn \log n) + O(kn) = O(mn + kn \log n)$.

3 Single-Source Shortest Paths

Let $G = (V, E)$ be a connected, undirected graph with positive edge lengths $\ell : E \to \mathbb{R}^+$. This graph represents a road network. You are trying to get from $s \in V$ to $t \in V$. Unfortunately, you are a coffee addict: getting caffeine often enough is the most important thing to you. In fact, there

is some number $L \in \mathbb{R}^+$ so that if you go more that L distance without getting coffee, you will get a caffeine migraine and crash your car. So you simply *cannot* go more than L distance without stopping for coffee. Fortunately, there is a coffee shop at every vertex $v \in V$ (including s and t) where you can get coffee. However, you are also a coffee snob, and you *hate* Starbucks. And there is a special subset of nodes $S \subseteq V$ where the only coffee shop on that node is a Starbucks.

So you have the following problem: given a subset $S \subseteq V$ of vertices where the only coffee shop is a Starbucks, find the path from s to t which uses only edges of length at most L, and has the *fewest* vertices in S. You may assume that $s, t \notin S$. Design an $O(m + n \log n)$-time algorithm for this problem, and prove running time and correctness.

Solution.

- **Step 1**
  firstly, Remove an edge whose weight exceeds L.

- **Step 2**
  Secondly, Change $\{x, y\}$ where $x, y \in V$ to $\{x, y\}$ and $\{y, x\}$. Afterward, set each weight as follow,
  $$w(x, y) = \begin{cases} 1 & \text{if } y \in S \text{ (the end node is a Starbucks),} \\ 0 & \text{otherwise.} \end{cases}$$

- **Step 3**
  Finally, run Dijkstra's algorithm from s on this new graph with weight w, and return the shortest path from s to t.

Correctness.

- **Step 1**
  By doing so, it is guaranted that you will not drive without coffee more than L distance.

- **Step 2**
  The original edges are undirected, but since we made the directed edges in both directions, every move along an original edge can still be represented in the new graph. Also, by the weight w, the weight of a path is exactly the number of times we enter a Starbucks node.

- **Step 3**
  All weights are 0 or 1, so they are non-negative and Dijkstra's algorithm works correctly. Therefore, the shortest path from s to t is the path whose number of visits to Starbucks nodes is minimum.

Running Time.

- **Step 1**
  We go through all edges once, which takes $O(m)$.

- **Step 2**
  We go through all edges once more and make $2m$ directed edges, which takes $O(2m) = O(m)$.

- **Step 3**
  Dijkstra's algorithm with a Fibonacci heap takes $O(m + n \log n)$, since there are n extract-min operations ($O(\log n)$ amortized each) and at most $2m$ decrease-key operations ($O(1)$ amortized each).

Therefore, the total running time is $O(m) + O(m) + O(m + n \log n) = O(m + n \log n)$.